Identification of image forgery using OpenCV

With the emergence and implementation of digital authentication in assorted applications, the identification of original user impressions is one of the key challenges today. Nowadays, many organisations ask for documents and certificates signed and scanned by applicants for self-attestation. This also happens when applying for new mobile SIM cards. However, imposters can take scanned signatures from any other document and put them in the required document using digital image editing tools like Adobe Photoshop, PaintBrush or PhotoEditors. There are a number of image editing tools available for the transformation of an actual image into a new image. This type of implementation can be (and often is) used for creating the new forged copy of the document, without the knowledge or permission of the actual applicant.

Figures 2 and 3 depict the process of forgery detection in a new document where the signatures are copied from another source.

The following source code written in Python and OpenCV presents the implementation of Flann based evaluation of images. OpenCV has enormous algorithms for the extraction of features in the images as well as in videos. Using such approaches, the manipulation in captured files can be identified and plotted.

```
import numpy as mynp
import cv2
from matplotlib import pyplot as plt
MIN_COUNT = 10
myimage1 = cv2.imread(''ChequewithCopiedSignature.png',0)
myimage2 = cv2.imread('OriginalCheque.png',0)
sift = cv2.xfeatures2d.SIFT_create()
# find the keypoints and descriptors with SIFT
k1, im1 = sift.detectAndCompute(myimage1,None)
k2, im2 = sift.detectAndCompute(myimage2,None)
FLANN_INDEX_KDTREE = 0
index_params = dict(algorithm = FLANN_INDEX_KDTREE, trees
= 5)
search_params = dict(checks = 50)
flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(im1,im2,k=2)
# store all the good matches as per Lowe's ratio test.
good = []
for m,n in matches:
    if m.distance < 0.7*n.distance:
        good.append(m)
if len(good)>MIN_COUNT:
    src_pts = mynp.float32([ k1[m.queryIdx].pt for m in good
]).reshape(-1,1,2)
    dst_pts = mynp.float32([ k2[m.trainIdx].pt for m in good
]).reshape(-1,1,2)
    M, mask = cv2.findHomography(src_pts, dst_pts, cv2.
RANSAC,5.0)
    matchesMask = mask.ravel().tolist()
    h,w = myimage1.shape
    pts = mynp.float32([ [0,0],[0,h-1],[w-1,h-1],[w-1,0]
]).reshape(-1,1,2)
    dst = cv2.perspectiveTransform(pts,M)
    myimage2 = cv2.polylines(myimage2,[mynp.
int32(dst)],True,255,3, cv2.LINE_AA)
else:
    print "Not enough matches are found - %d/%d" %
(len(good),MIN_COUNT)
    matchesMask = None
     draw_params = dict(matchColor = (0,255,0), # draw
matches in green color
                   singlePointColor = None,
                   matchesMask = matchesMask, # draw only
inliers
                   flags = 2)
myimg9 = cv2.drawMatches(myimage1,k1,myimage2,k2,good,None,*
*draw_params)
plt.imshow(myimg9, 'gray'),plt.show()
```

Figure 2: Scanned image of the original cheque

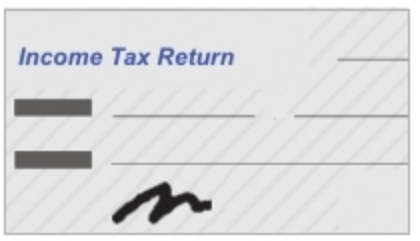

Figure 3: Signature used in another document by copying

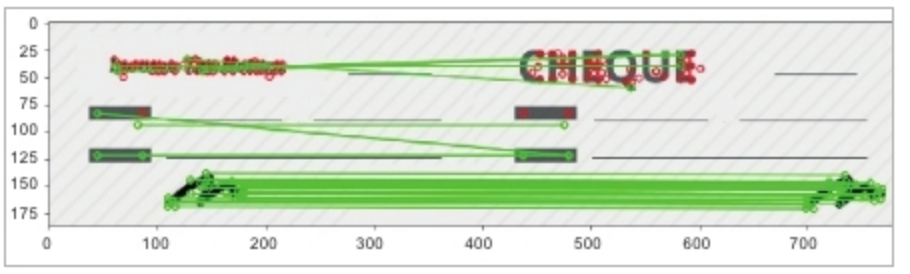

Figure 4: Identification of forged pixels in a cheque using OpenCV

It is evident from Figure 4 that the pixels have been identified in the new image (in which the signature has been copied from another source). So the marking of pixel values can be done using machine learning and inbuilt methods for prediction in OpenCV. Even if the person makes use of highly effective tools for image editing, the pixels from where the image segment is taken can be identified. END

By: Dr Gaurav Kumar and Amit Doegar

Dr Gaurav Kumar is the MD of Magma Research and Consultancy Services, Ambala. He is associated with various academic and research institutes, where he delivers lectures and conducts technical workshops. He can be reached at *kumargaurav.in@gmail.com*.

Amit Doegar is an assistant professor at the Department of Computer Science and Engineering, National Institute of Technical Teachers Training and Research (NITTTR). He can be contacted at *amit@nitttrchd.ac.in*.